

Competidor(a): _____

Número de inscrição: _____-_____ (opcional)

Este Caderno de Tarefas não pode ser levado para casa após a prova. Após a prova entregue este Caderno de Tarefas para seu professor guardar. Os professores poderão devolver os Cadernos de Tarefas aos competidores após o término do período de aplicação das provas (12 de setembro de 2025).



Olimpíada Brasileira de Informática

OBI2025

Caderno de Tarefas

Modalidade Programação • Nível 1 • Competição Feminina

12 de setembro de 2025

A PROVA TEM DURAÇÃO DE 3 HORAS

Promoção:



Sociedade Brasileira de Computação

Apoio:



Coordenação:



Instruções

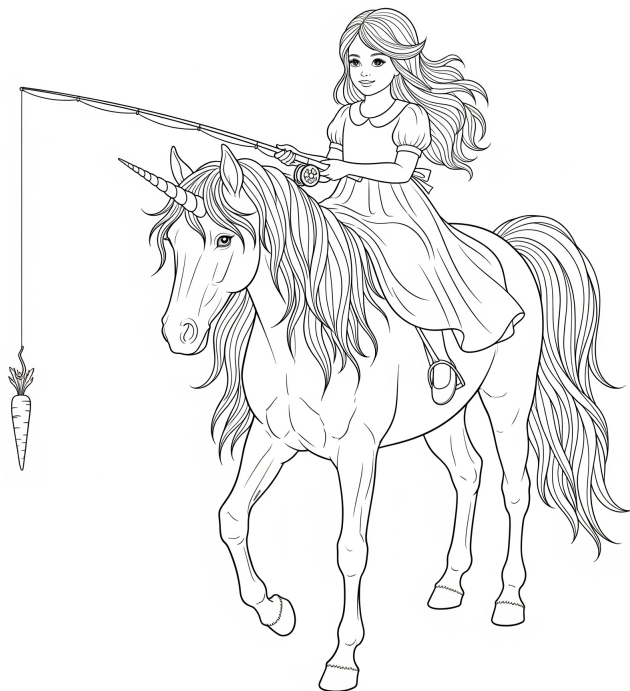
LEIA ATENTAMENTE ESTAS INSTRUÇÕES ANTES DE INICIAR A PROVA

- Este caderno de tarefas é composto por 9 páginas (não contando a folha de rosto), numeradas de 1 a 9. Verifique se o caderno está completo.
- A prova deve ser feita individualmente.
- É proibido consultar a Internet, livros, anotações ou qualquer outro material durante a prova. É permitida a consulta ao *help* do ambiente de programação se este estiver disponível.
- As tarefas têm o mesmo valor na correção.
- A correção é automatizada, portanto siga atentamente as exigências da tarefa quanto ao formato da entrada e saída de seu programa; em particular, seu programa não deve escrever frases como “Digite o dado de entrada:” ou similares.
- Não implemente nenhum recurso gráfico nas suas soluções (janelas, menus, etc.), nem utilize qualquer rotina para limpar a tela ou posicionar o cursor.
- As tarefas **não** estão necessariamente ordenadas, neste caderno, por ordem de dificuldade; procure resolver primeiro as questões mais fáceis.
- Preste muita atenção no nome dos arquivos fonte indicados nas tarefas. Soluções na linguagem C devem ser arquivos com sufixo *.c*; soluções na linguagem C++ devem ser arquivos com sufixo *.cc* ou *.cpp*; soluções na linguagem Pascal devem ser arquivos com sufixo *.pas*; soluções na linguagem Java devem ser arquivos com sufixo *.java* e a classe principal deve ter o mesmo nome do arquivo fonte; soluções na linguagem Python 3 devem ser arquivos com sufixo *.py*; e soluções na linguagem Javascript devem ter arquivos com sufixo *.js*.
- Na linguagem Java, **não** use o comando *package*, e note que o nome de sua classe principal deve usar somente letras minúsculas (o mesmo nome do arquivo indicado nas tarefas).
- Para tarefas diferentes você pode escolher trabalhar com linguagens diferentes, mas apenas uma solução, em uma única linguagem, deve ser submetida para cada tarefa.
- Ao final da prova, para cada solução que você queira submeter para correção, copie o arquivo fonte para o seu diretório de trabalho ou pen-drive, conforme especificado pelo seu professor.
- Não utilize arquivos para entrada ou saída. Todos os dados devem ser lidos da entrada padrão (normalmente é o teclado) e escritos na saída padrão (normalmente é a tela). Utilize as funções padrão para entrada e saída de dados:
 - em Pascal: *readln*, *read*, *writeln*, *write*;
 - em C: *scanf*, *getchar*, *printf*, *putchar*;
 - em C++: as mesmas de C ou os objetos *cout* e *cin*.
 - em Java: qualquer classe ou função padrão, como por exemplo *Scanner*, *BufferedReader*, *BufferedWriter* e *System.out.println*
 - em Python: *read*, *readline*, *readlines*, *input*, *print*, *write*
 - em Javascript: *scanf*, *printf*
- Procure resolver a tarefa de maneira eficiente. Na correção, eficiência também será levada em conta. As soluções serão testadas com outras entradas além das apresentadas como exemplo nas tarefas.

Unicórnio

Nome do arquivo: unicornio.c, unicornio.cpp, unicornio.pas, unicornio.java, unicornio.js ou unicornio.py

Vera é uma amazona que possui a dócil unicórnio Veraneio. Ambas estão se preparando para a corrida de velocidade mais famosa do Reino Encantado de Violeta. Vera possui uma estratégia muito esperta para que Veraneio fique incentivada a treinar: ela coloca na ponta de um anzol uma deliciosa cenoura que fica a 2 (dois) metros de distância à frente da unicórnio. Dessa forma, Veraneio fica muito animada para correr mais rápido.



Como auxiliar de treinamento desta astuta dupla, você deve criar um programa que, dada a posição horizontal X da Veraneio, responda qual é a posição da cenoura.

Entrada

A entrada possui uma única linha com o valor inteiro X que corresponde à posição horizontal da Veraneio, em metros.

Saída

A saída deve possuir uma única linha com a posição horizontal da cenoura que está à frente da Veraneio, em metros.

Restrições

- $0 \leq X \leq 10\,000$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas restrições adicionais às definidas abaixo.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (100 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1 0	Exemplo de saída 1 2
Exemplo de entrada 2 99	Exemplo de saída 2 101
Exemplo de entrada 3 234	Exemplo de saída 3 236

Copa do Mundo

Nome do arquivo: `copa.c`, `copa.cpp`, `copa.pas`, `copa.java`, `copa.js` ou `copa.py`

A paixão pelo futebol pulsa forte no coração brasileiro. A cada Copa do Mundo, o grito de “Brasil! Olê, Olê, Olê!” ecoa por todo o país. Inspirado nessa energia contagiante e na busca incessante pela glória, o problema que você enfrentará agora é uma homenagem às nossas conquistas mais memoráveis no maior palco do futebol.

E foi baseado nesse grito que, na Copa do Mundo de 2018, foi criada uma música de mesmo nome. A música fala sobre as cinco vitórias da nossa seleção brasileira ao longo dos anos, como nos trechos:

“É, em cinquenta e oito, foi Pelé
Em sessenta e dois, foi o Mané
Em setenta, o esquadrão
Primeiro a ser tricampeão

Oh, noventa e quatro, Romário
Dois mil e dois, Fenômeno
Primeiro tetracampeão
Único penta é o Brasilzão!”

Na letra da música, “cinquenta e oito”, “sessenta e dois”, “setenta”, “noventa e quatro” e “dois mil e dois” se referem, respectivamente, aos anos 1958, 1962, 1970, 1994 e 2002, que correspondem aos cinco anos nos quais o Brasil venceu a Copa do Mundo.

Dado um ano específico, você deve determinar se o Brasil venceu a Copa do Mundo e levantou a taça naquele ano.

Entrada

A entrada é composta por uma única linha contendo um único número inteiro N , representando o ano a ser verificado.

Saída

Seu programa deverá imprimir uma única linha contendo um único caractere. Caso o Brasil tenha ganhado uma Copa do Mundo naquele ano, imprima o caractere **S** (a letra S maiúscula). Caso contrário, imprima o caractere **N** (a letra N maiúscula).

Restrições

- $1930 \leq N \leq 2025$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas restrições adicionais, definidas a seguir.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados na tarefa e serve para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (15 pontos):** $N \geq 2010$.
- **Subtarefa 3 (21 pontos):** $N \geq 2000$.
- **Subtarefa 4 (64 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1 1958	Exemplo de saída 1 S
Exemplo de entrada 2 1982	Exemplo de saída 2 N

Campeonato de Xadrez

Nome do arquivo: `xadrez.c`, `xadrez.cpp`, `xadrez.pas`, `xadrez.java`, `xadrez.js` ou `xadrez.py`

Um incrível campeonato de xadrez está chegando na cidade, e Layse, que ama xadrez, quer participar. Ao buscar mais informações sobre o campeonato, Layse recebeu uma lista com a pontuação das N jogadoras que já se inscreveram antes dela. A lista está ordenada pela ordem de chegada, e ela contém as pontuações de cada jogadora que já se inscreveu.

Por exemplo, a lista $[800, 1600, 950, 1200, 1000, 1400]$ indica que:

- a primeira jogadora inscrita tem pontuação no xadrez de 800.
- a segunda jogadora inscrita tem pontuação de 1600.
- a terceira jogadora inscrita tem 950.
- e assim por diante

Para deixar as coisas mais emocionantes, a organizadora do evento decidiu que irá separar as participantes em K grupos contíguos pela ordem de chegada. As regras para a definição dos grupos serão:

- Uma jogadora só pode pertencer a apenas um grupo.
- Um grupo deve ter pelo menos uma jogadora.
- Se a jogadora A chegou antes da jogadora B e as duas estão no mesmo grupo, então todo mundo que chegou depois de A e antes de B também estará no mesmo grupo de A e B .
- Devem existir exatamente K grupos.

Contudo, como Layse não tem tanto tempo para treinar e vencer o campeonato, ela quer garantir que consegue vencer pelo menos M jogadoras de todos os grupos, para alguma divisão de grupos. Ou seja, se existir pelo menos uma divisão de grupos de forma que existam M jogadoras em cada grupo que Layse consiga vencer, então ela ficará feliz.

Em uma partida de xadrez entre duas jogadoras, vence aquela com a pontuação maior. Caso a pontuação seja a mesma, então ocorre um empate.

Por exemplo, se temos a lista de jogadoras $[800, 1600, 950, 1200, 1000, 1400]$, a organizadora formará 3 grupos, e Layse quer vencer pelo menos 1 pessoa de cada grupo, então a menor pontuação que ela precisaria ter é 1001. Considerando a seguinte divisão de grupos:

- grupo 1: $[800, 1600]$
- grupo 2: $[950]$
- grupo 3: $[1200, 1000, 1400]$

Isso é apenas uma de várias divisões possíveis para a mesma pontuação de Layse.

Portanto, ajude Layse a descobrir qual a menor pontuação necessária que faça ela feliz, ou seja, que ela consiga vencer no mínimo M jogadoras de cada grupo com as condições acima.

Entrada

A primeira linha da entrada contém três inteiros, N , M e K : a quantidade de jogadoras inscritas, quantas jogadoras Layse quer vencer e quantos grupos devem ser formados. A segunda linha contém N inteiros P_i , indicando a pontuação da jogadora na posição i , em ordem de chegada.

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, que será a pontuação mínima de Layse para que ela fique feliz. Imprima -1 caso não seja possível fazer Layse feliz.

Restrições

- $1 \leq M \leq N \leq 100\,000$
- $1 \leq K \leq N$
- $1 \leq P_i \leq 100\,000$

Informações sobre a pontuação

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (7 pontos):** Em um conjunto de teste valendo 7 pontos, $K = 1$.
- **Subtarefa 3 (14 pontos):** Em um conjunto de teste valendo 14 pontos, $1 \leq N \leq 100$, $K = 2$.
- **Subtarefa 4 (25 pontos):** Em um conjunto de teste valendo 25 pontos, $1 \leq N \leq 1\,000$.
- **Subtarefa 5 (54 pontos):** Em um conjunto de teste valendo 54 pontos, sem restrições adicionais.

Exemplos

Exemplo de entrada 1 6 1 3 800 1600 950 1200 1000 1400	Exemplo de saída 1 1001
Exemplo de entrada 2 5 2 5 100 200 300 400 500	Exemplo de saída 2 -1

Castelo de Blocos

Nome do arquivo: `castelo.c`, `castelo.cpp`, `castelo.pas`, `castelo.java`, `castelo.js` ou `castelo.py`

Durante uma tarde chuvosa, os irmãos Ana e Bruno decidiram construir um imenso castelo de blocos na sala de estar. Cada bloco foi cuidadosamente posicionado de modo que alguns sirvam de “base” para sustentar outros. Podemos representar cada bloco com um número de 1 a N , e dizer que se o bloco u serve de base para sustentar o bloco v , então v se apoia diretamente no bloco u , formando uma relação de apoio entre os blocos. Se um bloco w se apoia diretamente em v e v se apoia diretamente em u dizemos que w se apoia indiretamente em u . A estrutura desse castelo era muito particular: um bloco nunca se apoiava direta ou indiretamente em si mesmo, então não haviam ciclos de apoios.

Depois de horas de trabalho os irmãos decidiram que seria muito legal construir um segundo castelo, porém eles já tinha usado todos os blocos no primeiro! Por isso Bruno propôs o seguinte: ele iria retirar uma sequência específica de blocos $B_1, B_2, \dots, B_k$ (nesta ordem, sem repetições) do primeiro castelo para usarem na nova construção. Em cada passo i , Bruno tentará cuidadosamente retirar o bloco B_i do castelo.

No entanto, Ana reparou que um problema surge imediatamente após a remoção de cada bloco:

- Os blocos não são de boa qualidade, então qualquer bloco que perca todos os blocos em que se apoiava **diretamente** no castelo despenca e se quebra.
- A retirada ou quebra de um bloco pode causar novas quebras em cascata, ou seja, se um bloco é retirado ou quebra, os blocos que se apoiavam nele também podem ficar sem suporte e, consequentemente, quebrar.

Além disso, Ana também reparou que Bruno pode não conseguir retirar todos os blocos que deseja, já que alguns deles já poderiam ter sido quebrados nesse processo.

Sua tarefa é ajudar os irmãos a determinar, após ele seguir toda a sequência de retiradas, quantos blocos no total se quebraram nesse catastrófico processo!

Entrada

A primeira linha da entrada contém três inteiros N , M e K , representando respectivamente o número total de blocos no castelo dos irmãos, numerados de 1 a N , o número de relações de apoio entre os blocos e o número de blocos que Bruno planeja retirar.

As próximas M linhas contém dois inteiros u e v cada uma, indicando que o bloco v se apoia diretamente no bloco u .

A última linha contém K inteiros $B_1, B_2, \dots, B_k$, representando a sequência de blocos que Bruno tentará retirar, nesta ordem. Todos os B_i são distintos.

Saída

Seu programa deve imprimir um único inteiro: o número total de blocos que se quebraram ao final de todas as operações.

Restrições

- $1 \leq N \leq 100\,000$

- $1 \leq M \leq 200\,000$
- $1 \leq K \leq N$
- $1 \leq u, v \leq N$
- $1 \leq R_i \leq N$
- Todos os R_i são distintos
- As relações de apoio garantem que um bloco nunca se apoia em si mesmo, direta ou indiretamente, e não formam ciclos de dependência.

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas restrições adicionais às definidas acima.

- Subtarefa 1 (0 pontos): Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- Subtarefa 2 (6 pontos): $v = u + 1$, para todo par u, v
- Subtarefa 3 (10 pontos): $N \leq 100$
- Subtarefa 4 (24 pontos): $N \leq 1000$
- Subtarefa 5 (60 pontos): Sem restrições adicionais.

Exemplos

Exemplo de entrada 1 3 2 1 1 2 1 3 1	Exemplo de saída 1 2
Exemplo de entrada 2 4 3 2 1 2 2 3 2 4 2 3	Exemplo de saída 2 2
Exemplo de entrada 3 5 4 3 1 2 2 3 1 4 4 5 1 3 5	Exemplo de saída 3 4